

Hermes Agents vs LangChain Agents

Written against what this repo actually builds — a trade-compliance Researcher with real tools — and what the same system would look like in LangChain. Hermes claims cite the installed runtime (v0.18.2, MIT, Nous Research); LangChain claims verified against current docs, mid-2026. Token and reliability figures are measured on this repo, not estimated.

Hermes is a runtime you configure. LangChain is a library you assemble.

Both give you a tool-using agent quickly. They diverge on everything *around* the agent.

THE THREE DIMENSIONS

	HERMES	LANGCHAIN
Architecture	A runtime. One fixed ReAct loop (<code>AIAgent.run_conversation()</code>) serves CLI, HTTP gateway, cron and messaging alike. You configure what it can reach, not how it thinks. <i>Less to build, less to get wrong — and no way to express control flow the loop does not support.</i>	A library. <code>create_agent</code> is a prebuilt over LangGraph; underneath you define nodes, edges, cycles and interrupts yourself. You own the graph. <i>Expresses anything. You maintain it.</i>
Tool definition	An MCP server registered in <code>config.yaml</code>, or a plugin taking a plain callable. Schemas are generated from the function, so they cannot drift from it. Wrapped in injection scanning, a malware preflight and a filtered subprocess environment. <i>Tools live at the edge and cost nothing to anyone who does not enable them.</i>	<code>@tool / StructuredTool</code>, bound per agent with <code>.bind_tools()</code>. Same docstring-as-description idea. MCP supported via <code>langchain-mcp-adapters</code> — an add-on package rather than a config key. <i>Simplest possible to add; you supply any isolation yourself.</i>
State management	Four mechanisms, all built in, no external service: <code>MEMORY.md / USER.md</code> injected at session start; SQLite + FTS5 over every transcript (<code>session_search</code>, no LLM in the search path); Skills; and one optional external provider behind a single config key. <i>Persistence for free, on its terms — keyword rather than semantic recall by default.</i>	Deliberately separated, and you wire them: a LangGraph checkpoint for short-term (<code>SqliteSaver / PostgresSaver</code>), langgraph.store for long-term, usually surfaced through a tool you write. <i>No default, total choice — including vector recall. More work, more capability.</i>

WHAT THAT MEANS IN PRACTICE

	HERMES	LANGCHAIN
The agent loop	ships	ships
Sessions surviving a restart	ships	wire a checkpoint
Long-term memory	ships	wire a store + a tool to write with
A second agent	one config line (<code>delegate_task</code>)	build the graph
Swap the model	a four-line YAML file	a code change + a pip install
Web API with streaming	ships	write a server
Where the agent lives	<i>is</i> the process	embeds inside your app

For a two-tool demo, LangChain is less work. This exercise asked for long-term memory, a second agent and a model swap — which is exactly where the columns flip.

TWO THINGS WORTH SAYING OUT LOUD

Neither framework's value is the loop. The ReAct cycle is about fifteen lines. The value is everything around it you only discover you need once real users arrive — retries, truncation, session storage, search over it, memory policy, delegation.

A framework's defaults are the real cost. Hermes enables 49 tools by default, tuned for a general assistant. Scoping this agent to the five it uses cut the prompt from **21,373 to 6,694 tokens per call**, and tool-calling got *more* reliable — a model choosing among five routes better than one choosing among 49.

CHOOSING

Hermes when you want a sovereign, batteries-included runtime and your control flow fits a ReAct loop with delegation.

LangChain when the agent is a component inside a larger application, or you need graph topologies and retrieval infrastructure Hermes does not model.

Neither when you have two tools, one user and no memory requirement — that is the loop plus your functions, and a framework will cost you more in defaults than it saves you in code.

Both are MIT, and neither API is stable: LangChain has moved agent APIs twice in about a year (`AgentExecutor` → `create_react_agent` → `create_agent`), and Hermes is pre-1.0 — building this found its docs contradicting its own source in three places. **Pin versions and read the source, in either camp.**

Evidence — the loop written out, what a tool schema contains, the three routes into Hermes and why this repo took the third, and where open-weight models break — is in [hermes-vs-langchain-detail.md](#).